Deep Learning Algorithms and Implementations Report Understanding and Improving GPT-2 Implementations in NanoGPT

Wang Zeyu

1 Introduction

This report documents progress on understanding the multi-head self-attention implementation in NanoGPT and outlines the mathematical connection to the lecture slides. It also implemented code-level observations and plans to implement Key–Value (KV) caching for inference efficiency. Finally, it analyses the inference time and compares efficiency before and after implementing KV caching.

2 Task 1: Connecting Multi-head Attention Implementation in NanoGPT

We start from the notation used in the course slides and then map the symbols to the NanoGPT code variables.

Notation

We adopt the slide notation and extend it where necessary:

- B : batch size (number of sequences processed in parallel).
- T : sequence length (number of tokens in a sequence).
- d (or C) : model embedding dimensionality (also `n_embd` in code).
- h (or nh) : number of attention heads.
- d_h (or hs) : head dimension, $d_h = d/h$.
- $\tilde{Z} \in \mathbb{R}^{T \times d}$: input matrix/ tensor in slides (**single** sequence).
- $x \in \mathbb{R}^{B \times T \times d}$: input tensor in code (**batch**).

2.1 Slides Overview

The lecture slides introduce single-head attention as:

$$\text{Attention}(Q, K, V) \;=\; \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where projections are:

$$Q = \tilde{Z}W_Q, \quad K = \tilde{Z}W_K, \quad V = \tilde{Z}W_V,$$

where typically $W_Q, W_K, W_V \in \mathbb{R}^{d \times d'}$ (for single-head often $d' = d$). For multi-head attention the slides express, for head i,

$$\text{head}_i = \text{Softmax}\left(\frac{\tilde{Z}W_Q^{(i)}(\tilde{Z}W_K^{(i)})^\top}{\sqrt{d_h}}\right) \tilde{Z}W_V^{(i)},$$

and then

$$\text{MultiHead}(\tilde{Z}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O,$$

From here on, we shall see the differences between what was taught in the slides above and implementation in NanoGPT.

2.2 Computation of Q, K, and V

2.2.1 Difference 1: Combined projection

```python
# in model.py (CausalSelfAttention.forward)
# q, k, v generation
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
```

NanoGPT combines the 3 projections into a single linear projection that computes all 3 at once. Mathematically we can write a concatenated projection matrix:

$$W_{\text{combined}} \;=\; [W_Q \mid W_K \mid W_V] \in \mathbb{R}^{d \times 3d},$$

At the same time, the bias terms are also concatenated/combined:

$$b_{\text{combined}} \;=\; [b_Q \mid b_K \mid b_V] \in \mathbb{R}^{3d},$$

Now, instead of $\tilde{Z}$, NanoGPT uses x, because x includes B, the batch size:

$$x \in \mathbb{R}^{B \times T \times d}, \quad W_{\text{combined}} \in \mathbb{R}^{d \times 3d}, \quad b_{\text{combined}} \in \mathbb{R}^{3d},$$

$$\text{out} \;=\; x \cdot W_{\text{combined}} \;+\; b_{\text{combined}}, \qquad \text{out} \in \mathbb{R}^{B \times T \times 3d},$$

Why Combine the projections?

At first glance, this seems mathematically off. The matrix multiplication of x (3D) and $W_{combined}$ (2D), and somehow adding a $b_{combined}$ (1D) defies linear algebra. However, in PyTorch, they are using batched matrix multiplication and broadcasting, applying the multiplication to each (B, T) vector and adding the bias across all $(B, T, 3d)$ positions, therefore, we can do operations with different dimensions. The reason is that we enable **parallel computations** that is more efficient than computing sequentially like in the slides.

In the next step, we split along the embedding dimension (the last dim), into the Q,K,V:

$$Q, K, V \;=\; \text{split}(\text{out};\, d, d, d),$$

so that

$$Q = xW_Q + b_Q, \quad K = xW_K + b_K, \quad V = xW_V + b_V, \quad \text{where } Q, K, V \in \mathbb{R}^{B\times T\times d}.$$

This combined projection allows all three linear transformations to share a single matrix multiplication. Thanks to the batched matrix operations in PyTorch, it reduces memory overhead and improves GPU efficiency.

2.3 Head Splitting and Tensor Reshaping

2.3.1 Difference 2: Reshaped Tensors

```
# head splitting and reshaping
k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
```

After obtaining $Q, K, V \in \mathbb{R}^{B\times T\times d}$, NanoGPT reshapes for multi-head attention. The mathematical equivalent of the code operations: (Python indexing)

$$\begin{aligned}
&\textbf{view:} \quad Q \in \mathbb{R}^{B\times T\times d} \quad\longrightarrow\quad Q' \in \mathbb{R}^{B\times T\times h\times d_h},\\
&\textbf{transpose:} \quad \text{transpose}_{1,2}(Q') \longrightarrow Q'' \in \mathbb{R}^{B\times h\times T\times d_h}.
\end{aligned}$$

So for each head i we obtain

$$Q^{(i)} \in \mathbb{R}^{B\times T\times d_h}, \quad K^{(i)} \in \mathbb{R}^{B\times T\times d_h}, \quad V^{(i)} \in \mathbb{R}^{B\times T\times d_h},$$

and attention per head is computed (batch-wise) as

$$\text{head}_i = \text{Softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_h}}\right) V^{(i)}, \quad \text{head}_i \in \mathbb{R}^{B\times T\times d_h}$$

Why reshape / transpose?

Reshaping: splits the d-dimensional embedding into h heads, with size d_h each: this gives the per-head projection matrices that the slides describe.

$$Q \in \mathbb{R}^{B\times T\times d} \longrightarrow Q' \in \mathbb{R}^{B\times T\times h\times d_h}.$$

Transposing: swaps the head and sequence dimensions:

$$\text{transpose}_{1,2}(Q') \longrightarrow Q'' \in \mathbb{R}^{B\times h\times T\times d_h}$$

By transposing to (B, h, T, d_h), the head dimension h is now treated like an extra batch dimension. PyTorch interprets the tensor as $B \cdot h$ independent "batches" of size (T, d_h). Without this, each head's attention would have to be computed sequentially in a Python loop:

$$\text{for } i = 1, \ldots, h: \quad \text{head}_i = \text{Softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_h}}\right) V^{(i)}.$$

Instead, after transposing, PyTorch interprets (B, h, T, d_h) as $B \cdot h$ independent "batches" of size (T, d_h), enabling batched attention:

$$\text{head}_{all} = \text{Softmax}\left(\frac{Q''(K'')^\top}{\sqrt{d_h}}\right) V''$$

This is executed in a single GPU kernel call via batched matrix multiplication, computing attention for all heads in parallel. This eliminates Python-level loops and maximises GPU parallelism and memory efficiency.

3 Task 2: Implement and evaluate KV Caching to NanoGPT with a focus on inference efficiency

Here, we try to implement KV caching by editing `model.py`, modifying the `forward` function to accept a `past` key-value cache that stores $\boldsymbol{k}_1$ to $\boldsymbol{k}_{T-1}$ and $\boldsymbol{v}_1$ to $\boldsymbol{v}_{T-1}$.

By looking at `sample.py` and `model.py` in GPT-2, we see how KV caching is implemented, which gives us clues for implementing it in NanoGPT. In GPT-2, this is implemented as follows:

3.1 KV Caching in GPT-2

KV caching stores previously computed key and value tensors to avoid recomputation during autoregressive inference. In GPT-2, this is implemented as:

```
def attn(x, scope, n_state, *, past, hparams):
    assert x.shape.ndims == 3  # Should be [batch, sequence, features]
    assert n_state % hparams.n_head == 0
    if past is not None:
        assert past.shape.ndims == 5  # [batch, 2, heads, sequence, features], where 2 is [k, v]
```

The attention function accepts an optional argument `past`, which contains the cached keys and values:

$$\text{past} = (\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{T-1},\ \boldsymbol{v}_1, \ldots, \boldsymbol{v}_{T-1}).$$

There is an extra dimension on axis = 1, where it helps to differentiate the keys and values within the argument `past`.
The main caching logic occurs here, within the `attn` function:

```
    if past is not None:
        pk, pv = tf.unstack(past, axis=1)
        k = tf.concat([pk, k], axis=-2)
        v = tf.concat([pv, v], axis=-2)
```

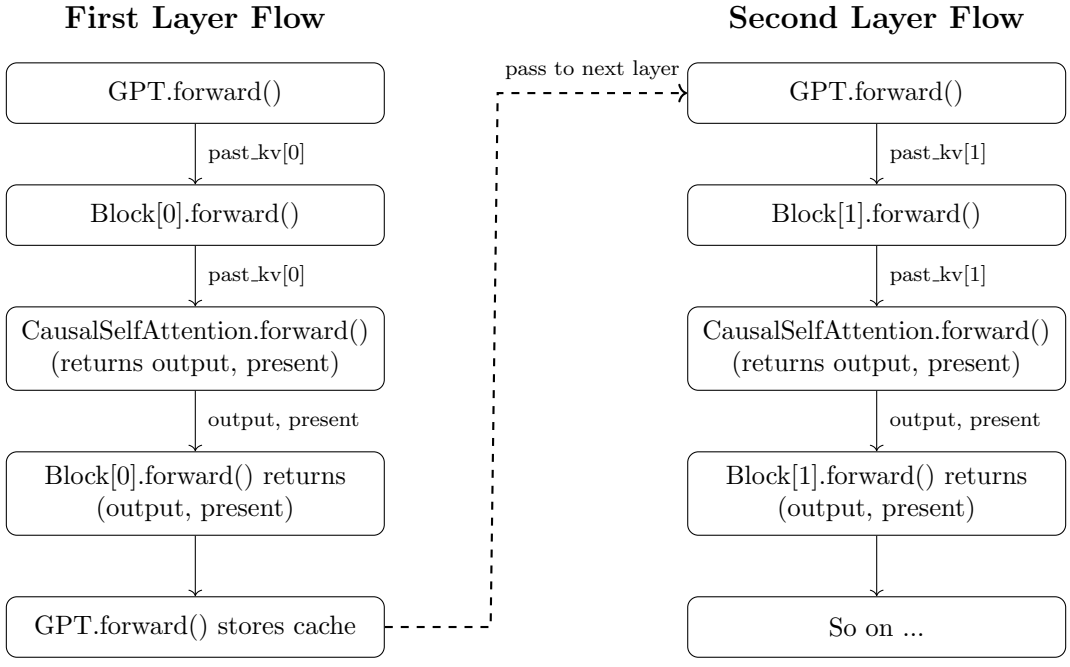
This checks if a past cache exists or not (first token or not). If yes, the model concatenates the new keys and values with the cached ones, allowing the attention computation to reuse $\boldsymbol{k}_1 \ldots \boldsymbol{k}_{T-1}$ and $\boldsymbol{v}_1 \ldots \boldsymbol{v}_{T-1}$ without recomputing them. This happens by:

- split `past` into past keys (`pk`) and past values (`pv`)
- concatenate the new key and value for the current token with the cached ones (`pk`,`pv`) on `axis = -2` (sequence)

Therefore, we need to do something similar. In NanoGPT, we must modify the `forward` function to accept an optional `past` argument that stores the previously computed key and value tensors for each attention head. During inference, the model will then concatenate the cached KV with the new KV.

3.2 KV caching implementation on NanoGPT

KV caching requires the cache to flow through the entire model stack. Each layer reuses cached kv. This flow is illustrated below:



Key Insight: To support KV caching efficiently, we need to modify the `forward` method in all three classes that contains the `forward` method:

1. `CausalSelfAttention`: accept `past_kv` and return `present` cache.
2. `Block`: accept `past_kv`, forward it to its attention module, and return `present`.
3. `GPT`: manage a list of `past_kv` for each layer and propagate it correctly during inference.

3.2.1 CausalSelfAttention.forward Method

Change 1: Logic for KV caching

```
def forward(self, x, past_kv=None):
    B, T, C = (x.size())
...
    # CHANGE: added LOGIC for kv_caching, literally the same as GPT 2
    if past_kv is not None:
        past_k, past_v = past_kv
        k = torch.cat([past_k, k], dim=2)
        v = torch.cat([past_v, v], dim=2)
    present = (k, v)  # the new cache to return
```

• **Added past_kv argument:** None if this is the first token, and subsequent ones shall be updated.
• **Logic for kv caching:** Exactly the same logic as GPT-2 as above. Not going to repeat.

Change 2: Causal Masking

```
    # Self-attend: (B, nh, T, hs) x (B, nh, hs, kv_len) -> (B, nh, T, kv_len)
    if self.flash:
        # CHANGE: only turn on causal if there is no past_kv. No longer need causal masking for the entire
        ↪  sequence
        is_causal = past_kv is None
        y = torch.nn.functional.scaled_dot_product_attention(...is_causal=is_causal,  # CHANGE: no longer
        ↪  always true)
    else:
        # manual implementation of attention
        if past_kv is not None:
            # SKIP mask application when using cache
            pass
        else:
            att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))
        att = F.softmax(att, dim=-1)
        att = self.attn_dropout(att)
        y = att @ v  # (B, nh, T, kv_len) x (B, nh, kv_len, hs) -> (B, nh, T, hs)
    y = (
        y.transpose(1, 2).contiguous().view(B, T, C)
    )  # re-assemble all head outputs side by side
...
    return y, present
```

• **Causal Masking Explanation:**
Without KV cache, causal masking is required. An example is shown to illustrate:

```
Input: [A, B, C]  # All processed in parallel in one forward pass
# Attention without masking:
A -> [A, B, C]  # A sees future tokens B, C
B -> [A, B, C]  # B sees future token C
C -> [A, B, C]  # C sees only past
```

Since the inputs are processed in parallel, A can now "see" B and C, which violates causality. Masking **prevents each token from attending beyond its own position**.

With KV cache, only the new token is processed:

```
Cache: [A, B, C]  # Cached keys/values from previous steps
Input: [D]        # Only new token processed
D → [A, B, C, D]  # No need for masking
```

Since only one token queries and all attended positions are $\leq$ its position, no masking required. We therefore only do causal masking for the first prompt (forward pass). In the code, when `past_kv` is not None (second pass onwards):

1. Disable Flash Attention's built-in causal masking
2. Skip manual mask application

• **Dimension Changes:** With KV caching, the key and value tensors now include the cached past sequence.

- The sequence length dimension of $\boldsymbol{k}, \boldsymbol{v}$ changes from T to $T_{past} + T$, where T_{past} is the length of the cached sequence.
- Therefore, $\boldsymbol{k}, \boldsymbol{v}$ change shape from (B, nh, T, hs) to $(B, nh, T_{past} + T, hs)$.
- Consequently, the attention matrix changes from (B, nh, T, T) to $(B, nh, T, T_{past} + T)$.
- The output y still has shape (B, nh, T, hs) because attention is only computed for the current input tokens.

3.2.2 Block.forward Method

Change 1: Pass and Return KV cache

```python
# CHANGE: forward now accepts and returns past_kv
def forward(self, x, past_kv=None):
    # Self-attention returns output and updated KV cache
    y, present = self.attn(self.ln_1(x), past_kv=past_kv)
    # same as before
    x = x + y
    # Feedforward
    x = x + self.mlp(self.ln_2(x))
    return x, present
```

• **Added `past_kv` parameter:** Accepts the cached kv from the previous generation step.

• **Unpacking attention output:** `self.attn` now returns `(y, present)` due to modified `CausalSelfAttention`'s output as `(y, present)`.
The residual connection logic (`x = x + y`) remains unchanged, its just that the logic is split into 2 lines.

3.2.3 GPT.forward Method

Change 1: Absolute Position Tracking

```python
def forward(self, idx, targets=None, past_kv=None):
    ...
    # CHANGE: when there is kv cache, the position stored changes! so we need to get their absolute position
    if past_kv is not None and past_kv[0] is not None:
        past_length = past_kv[0][0].size(2)  # get the length of the past sequence (past_seq_len) as shown
        ↪  here (B, nh, past_seq_len, hs)
        # now the range of values starts with past_length
        pos = torch.arange(
            past_length, past_length + t, dtype=torch.long, device=device
        )
    else:
        pos = torch.arange(0, t, dtype=torch.long, device=device)
```

Explanation: Tokens need position embeddings corresponding to their *absolute position* when doing kv caching, because there is a mismatch in indexing. To explain with an example:
Example: Predicting `"coolest"` after prompt `["I", "am", "the"]`
Without KV cache (reprocess all tokens):

```
idx = ["I", "am", "the", "coolest"]
pos = [0, 1, 2, 3]
pos_emb = wpe([0, 1, 2, 3]) # absolute positions match the embedded positions
```

The absolute positions match with the position embeddings without kv cache. **With KV cache**(only process first token):

```
# "coolest" is at absolute position 3, but...
idx = ["coolest"]           # Only new token (length=1)
pos = [0]               # Would use position 0, WRONG!
pos_emb = wpe([0])      # Embedding for position 0, not 3!
```

The new token `"coolest"` have absolute position of 3, but the embedded position is 0. To match the absolute position to the embedded position, we add `past_length` (number of cached tokens) to the current position indices.

Change 2: Updating Present kv

```python
    present_kvs = []
    for i, block in enumerate(self.transformer.h):
```

```
        # feed correct cache to each layer by getting layer i's old cache
        layer_past = past_kv[i] if past_kv is not None else None

        # block returns hidden state + kv
        x, layer_present = block(x, past_kv=layer_past)
        present_kvs.append(layer_present)
        ...
```

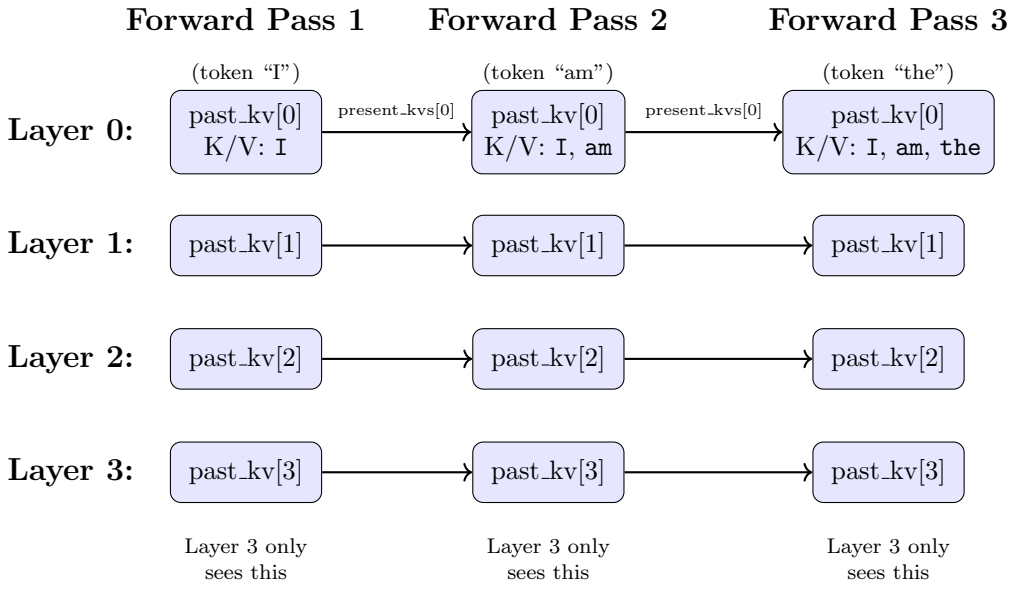
Why each layer needs its own cache: Each transformer layer applies different learned transformations, producing unique kv representations. These must be cached separately per layer.

The flow:

1. Initialise `present_kvs` that holds up to `n_layers`, which is determined by model type
2. `layer_past = past_kv[i]` fetches the cached kv from layer `i`'s previous forward pass
3. Block concatenates cached kv with new kv, performs attention, returns updated cache
4. `present_kvs.append(layer_present)` stores the updated cache for layer `i`
5. `present_kvs` becomes `past_kv` for the next token generation

Without this, each layer would recompute attention over all previous tokens instead of reusing cached computations.

Visualisation of cache flow across forward passes:

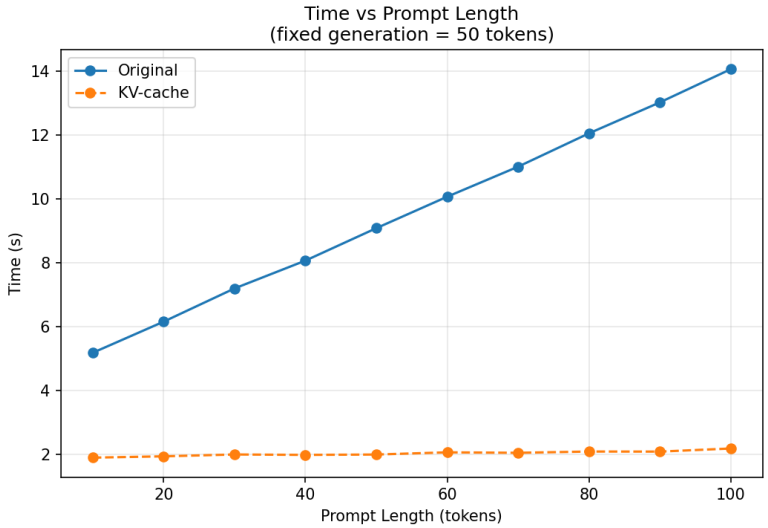


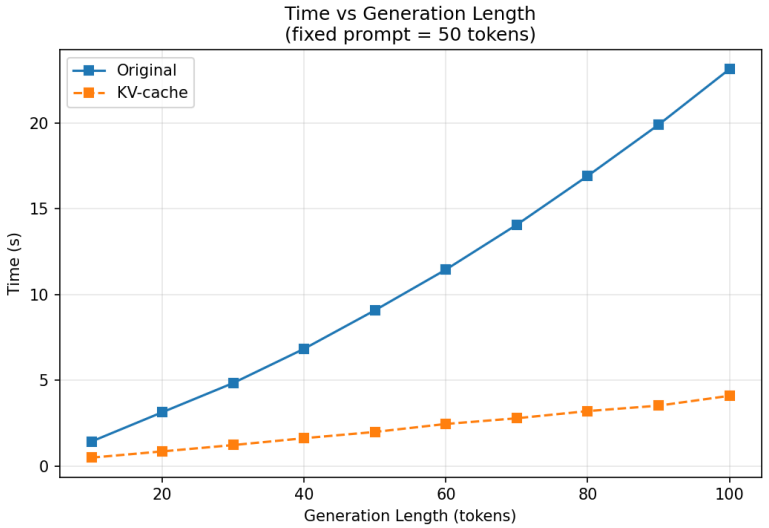
The diagram illustrates how each layer maintains its own cache across multiple forward passes. Notice that each layer's cache only flows horizontally to itself in subsequent passes, never mixing with other layers' caches.

We are now done with the implementation, let's take a look at the performance with and without kv caching.

3.3 Benchmark differences with and without KV caching

We plot 2 graphs below: Time vs Prompt Length/ No. of Input Tokens (fixed generation = 50) and Time vs Generation Length (fixed prompt=50).
We are trying to evaluate how inference time scales with prompt length and generation length before and after KV caching.





From the graphs, we can observe significant improvements after implementing KV caching:

- **Time vs Prompt Length:**
  - **Without KV caching**, the generation time grows linearly with prompt length, because the model must **recompute** attention over the entire prompt plus all previously generated tokens.
  - **With KV caching**, the generation time remains nearly constant (∼2s) regardless of prompt length, because the prompt's kv pairs are computed once and **reused** throughout generation.
- **Time vs Generation Length:**
  - **Without KV caching**, the generation time grows quadratically ($O(n^2)$) where n is the number of generated tokens. Each new token requires recomputing attention over all previous tokens (prompt + generated). For generating n tokens, this leads to $1 + 2 + 3 + ... + n = \frac{n(n+1)}{2}$ operations, which is dominated by the $O(n^2)$ term.
  - **With KV caching**, the time grows linearly ($O(n)$) where n is the number of generated tokens, because cached KV pairs are reused and only the new token's attention needs to be computed at each step, resulting in n operations total (one per generation step).

4 Conclusion

In this report, we have documented the following:

1. **Mathematical vs. computational efficiency:** While the underlying mathematical definitions of attention and autoregressive generation remain the same, differences in implementation can lead to improvements in computational efficiency.

   By reorganising computations and using parallel computation, modern architectures can perform the same operations much faster.

2. **KV caching in NanoGPT:** We implemented KV caching in NanoGPT by modifying the `forward` methods in `model.py`. This required modification of logic and indexing to correctly reuse cached kv pairs.

   Our experiments demonstrate that KV caching significantly improves inference efficiency: generation time increases much more slowly with longer prompts or larger generation lengths, compared to the original implementation without caching.

Key takeaway: The benchmark clearly demonstrates that KV caching drastically improves inference efficiency in GPT models. After KV caching implementation, generation time becomes largely independent of the input prompt length and grows only linearly with the number of generated tokens, making KV caching essential for modern large-scale text generation.

References

[1] Andrej Karpathy, *nanoGPT*, `https://github.com/karpathy/nanoGPT`, 2023.

[2] OpenAI, *GPT-2: Language Models are Unsupervised Multitask Learners*, `https://github.com/openai/gpt-2`, 2019.